

1. Gestión de Categorías de Productos (El "CRUD")

Es muy probable que te pidan gestionar las ProductCategory. Como el backend ya tiene los endpoints, tú tendrás que hacer la interfaz.

- **Lo que te pedirán:**
 - "Crea una pantalla que liste las categorías de un restaurante."
 - "Añade un botón para crear una nueva categoría."
 - "Permite editar o borrar una categoría existente."
 - **Cómo te lo preguntarán:**
"Implemente la funcionalidad necesaria para que un propietario pueda gestionar las categorías de sus productos. La pantalla debe mostrar el nombre de la categoría y el número de productos asociados."
-

2. Lógica de Negocio y Estados (Órdenes)

Suelen pedirte trabajar con los estados de los pedidos (Pending, In Process, Sent, Delivered).

- **Lo que te pedirán:**
 - "Muestra los pedidos de un restaurante (OrdersScreen)."
 - "Implementa el botón 'Advance' para pasar al siguiente estado."
 - "Haz que el botón desaparezca si el pedido ya está entregado."
 - **Cómo te lo preguntarán:**
"Modifique la pantalla de pedidos para que el usuario pueda avanzar el estado de una orden. Asegúrese de que la interfaz se refresque automáticamente y que el botón de avance no sea visible en pedidos con estado 'delivered'."
-

3. Analíticas y Estadísticas

Es un clásico pedirte una pequeña sección de "Dashboard" con datos calculados por el backend.

- **Lo que te pedirán:**
 - "Muestra la facturación del día y el número de pedidos pendientes."
 - "Usa un color diferente (rojo/verde) según si hay pedidos pendientes o no."
 - **Cómo te lo preguntarán:**
"Incluya un panel de analíticas en la cabecera de la pantalla de pedidos que muestre el importe total facturado hoy (formateado a dos decimales) y el conteo de pedidos pendientes."
-

4. Validación de Formularios (Formik + Yup)

No se conforman con que el formulario funcione; tiene que ser robusto.

- **Lo que te pedirán:**
 - "Valida que el nombre no esté vacío."
 - "Valida que el precio sea un número positivo."
 - "Muestra los errores que devuelve el servidor (Backend Errors)."
 - **Cómo te lo preguntarán:**

"Cree el formulario de edición de pedidos asegurándose de que todos los campos obligatorios estén validados mediante Yup. Si el servidor devuelve un error de validación, este debe mostrarse debajo del campo correspondiente."
-

5. Navegación y Paso de Parámetros

Suelen evaluar si sabes moverte entre pantallas enviando los IDs correctos.

- **Lo que te pedirán:**
 - "Desde la pantalla de detalle del restaurante, añade un acceso a la gestión de categorías."
 - "Pasa el restaurantId como parámetro."
-



Consejos Técnicos para estar Preparado

Para no quedarte en blanco durante el examen, ten dominado esto:

1. **El Hook useEffect:** Úsalo siempre para cargar datos al entrar (fetchData). Recuerda pasarle [route] o un array vacío [] para que no haga bucles infinitos.
2. **FlatList con ListHeaderComponent:** Es la mejor forma de poner las estadísticas (Analytics) arriba y los pedidos abajo en una sola lista con scroll.
3. **Operador Ternario y Cortocircuito:** * {item.status !== 'delivered' && <Button />} (para mostrar/ocultar).
 - style={item.length === 0 ? styles.red : styles.green} (para colores dinámicos).
4. **Los Endpoints:** Memoriza o ten a mano cómo se llaman las funciones en tu carpeta api/. Por ejemplo, getDetail, update, create.

Ejemplo de pregunta "trampa":

"Haga que la lista de categorías se refresque cuando el usuario vuelva de la pantalla de creación."

- **Solución:** Al navegar de vuelta, usa navigation.navigate('Pantalla', { dirty: true }) y en el useEffe

1. El Indicador Visual (En RestaurantsScreen)

Te pedirán que en la tarjeta de cada restaurante aparezca su estado actual de saturación.

- **La pregunta:** "Modifique la tarjeta del restaurante para que muestre un aviso de 'Saturado' en rojo si el restaurante no es ilimitado y ha recibido pedidos en las últimas 2 horas. En caso contrario, muestre 'Disponible' en verde."
- **Cómo implementarlo:** Usarás un **renderizado condicional** de estilos.

```
// Dentro de renderRestaurant
const isSaturated = !item.isUnlimited && item.ordersInLastTwoHours > 0

return (
  <View style={styles.card}>
    <TextRegular textStyle={{ color: isSaturated ? 'red' : 'green' }}>
      {isSaturated ? 'Estado: SATURADO' : 'Estado: DISPONIBLE'}
    </TextRegular>
    <TextRegular>Pedidos (últimas 2h): {item.ordersInLastTwoHours}</TextRegular>

    {item.isUnlimited &&
      <MaterialCommunityIcons name="infinity" size={20} color="gold" />
    }
  </View>
)
```

2. El Switch con Restricción (En EditRestaurantScreen)

Aquí evaluarán si sabes usar un componente Switch y cómo gestionar errores que vienen del servidor.

- **La pregunta:** "Añada un campo Switch al formulario de edición de restaurante para marcarlo como 'ilimitado'. Tenga en cuenta que el sistema solo permite 3 restaurantes ilimitados por propietario; si el servidor devuelve un error, muéstrelo al usuario."
- **La clave técnica:** El límite de 3 suele validarlo el **backend**. Tu trabajo es capturar ese error en el catch y mostrarlo con showMessage o setBackendErrors.

```
// En el Formik
<View style={styles.switchRow}>
  <TextRegular>¿Restaurante Ilimitado?</TextRegular>
  <Switch
    value={values.isUnlimited}
    onChange={value => setFieldValue('isUnlimited', value)}
  />
</View>

// En el submit (catch)
} catch (error) {
  // Si el backend devuelve: "Maximum 3 unlimited restaurants allowed"
  setBackendErrors(error.errors)
}
```

3. Lógica de "Bloqueo" (UX)

Te pueden pedir que si un restaurante está saturado, se le impida al usuario hacer ciertas acciones (como crear nuevos productos o recibir pedidos).

- **La pregunta:** *"Si un restaurante está saturado, el botón de 'Crear Producto' debe aparecer deshabilitado y en color gris para evitar sobrecargar el sistema."*
- **Cómo implementarlo:**

```
<Pressable
  disabled={isSaturated}
  style={[
    styles.button,
    { backgroundColor: isSaturated ? 'gray' : GlobalStyles.brandPrimary }
  ]}
>
  <Text>Crear Producto</Text>
</Pressable>
```

1. Indicadores de Saturación (Visualización)

Si en el backend has creado `isClosedByLimit` y `ordersInLastTwoHours`, en el frontend te pedirán que esto se vea en la lista de restaurantes (`RestaurantsScreen`).

- **La pregunta:** "Muestre un badge o texto que indique si el restaurante está 'Saturado' (`isClosedByLimit`) y cuántos pedidos ha recibido en las últimas 2 horas."
- **Lo que tienes que hacer:**
 - Usar un **renderizado condicional** para el color (rojo si está cerrado, verde si no).
 - Mostrar el contador que viene de la API.

```
// Dentro del renderItem de RestaurantsScreen
<View style={styles.statusBadge}>
  <TextRegular textStyle={{ color: item.isClosedByLimit ? 'red' : 'green' }}>
    {item.isClosedByLimit ? '🚫 Saturado' : '✅ Disponible'}
  </TextRegular>
  <TextRegular>Pedidos (2h): {item.ordersInLastTwoHours}</TextRegular>
</View>
```

2. Conmutar el estado "ilimitado" (PATCH)

Como creaste el endpoint PATCH `/restaurants/:restaurantId/toggleIsUnlimited`, lo lógico es que te pidan un botón para usarlo, probablemente en `MyRestaurantsScreen` o en el detalle.

- **La pregunta:** "Añada un botón o Switch para conmutar si un restaurante es ilimitado. El botón debe reflejar el estado actual (`isUnlimited`)."
- **Lo que tienes que hacer:**
 - Crear la función en `api/RestaurantEndpoints.js` que llame al PATCH.
 - Implementar un `Pressable` que llame a esa función y luego refresque la lista.

3. Gestión del Error 409 (Validación de Negocio)

Este es un punto donde suelen "pillar" a los alumnos. En el backend devuelves un **409 Conflict** si ya hay 3 ilimitados. En el frontend debes capturar ese error específico.

- **La pregunta:** "Si al intentar marcar un restaurante como ilimitado el servidor devuelve un error de conflicto (409), muestre un mensaje de error específico al usuario indicando que ya tiene el cupo máximo de 3."
- **Lo que tienes que hacer:**
 - En el catch de tu función, mirar el código de error o el mensaje que viene del backend.

```

const handleToggle = async (id) => {
  try {
    await toggleIsUnlimited(id)
    fetchRestaurants() // Refrescar
  } catch (error) {
    const message = error.status === 409
      ? 'Ya tienes 3 restaurantes ilimitados. Desactiva uno primero.'
      : 'Error al cambiar el estado.'

    showMessage({ message, type: 'danger' })
  }
}

```

4. Filtro o Ordenación por Saturación

A veces piden que los restaurantes saturados aparezcan al final de la lista o que tengan una opacidad menor para indicar que "no se puede pedir".

- **La pregunta:** *"Haga que los restaurantes que estén cerrados por límite (isClosedByLimit) aparezcan con una opacidad del 50% y el botón de 'Crear Pedido' esté deshabilitado."*